

Flutter 動畫

來源：<https://codelabs.developers.google.com/advanced-flutter-animations?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何在 Flutter 中使用動畫。您將建立小工具，為大小和顏色製作動畫、加入 3D 卡片翻轉效果、使用動畫套件中的特效，以及新增 Android 的預測返回手勢支援功能。

目錄

1. [1. 簡介](#)
2. [2. 設定 Flutter 開發環境](#)
3. [3. 執行範例應用程式](#)
4. [4. 使用隱含動畫效果](#)
5. [5. 使用明確的動畫效果](#)
6. [6. 控制動畫狀態](#)
7. [7. 使用自訂導覽轉換](#)
8. [8. 恭喜](#)

1. 簡介

動畫是提升應用程式使用者體驗的絕佳方式，可向使用者傳達重要資訊，並讓應用程式更精緻，使用起來也更愉快。

Flutter 動畫架構總覽

Flutter 會在每個影格重建部分小工具樹狀結構，顯示動畫效果。提供預先建構的動畫效果和其他 API，讓您更輕鬆地建立及組合動畫。

- **隱含動畫**是預先建構的動畫效果，可自動執行整個動畫。當動畫的**目標值**變更時，動畫會從目前值執行至目標值，並顯示中間的每個值，讓小工具順暢地呈現動畫效果。隱性動畫的範例包括 `AnimatedSize`、`AnimatedScale` 和 `AnimatedPositioned`。
- **明確動畫**也是預先建構的動畫效果，但需要 `Animation` 物件才能運作。例如：`SizeTransition`、`ScaleTransition` 或 `PositionedTransition`。
- **Animation** 類別代表正在執行或已停止的動畫，由代表動畫執行目標值的 *value*，以及代表動畫在任何指定時間顯示於螢幕上的目前值的 *status* 所組成。這是 `Listenable` 的子類別，會在動畫執行期間狀態變更時通知監聽器。
- **AnimationController** 可用來建立動畫及控制動畫狀態。您可以使用 `forward()`、`reset()`、`stop()` 和 `repeat()` 等方法控制動畫，不必定義顯示的動畫效果，例如縮放、大小或位置。
- **補間動畫**用於**內插**開始值和結束值之間的數值，且可代表任何型別，例如 `double`、`Offset` 或 `Color`。
- **曲線**可用於調整參數隨時間變化的速率。執行動畫時，通常會套用**緩和曲線**，在動畫開始或結束時加快或減慢變化率。曲線會接受介於 0.0 和 1.0 之間的輸入值，並傳回介於 0.0 和 1.0 之間的輸出值。

建構項目

在本程式碼研究室中，您將建構選擇題測驗遊戲，並加入各種動畫效果和技巧。



您將瞭解如何...

- 建立可產生大小和顏色動畫的小工具
- 建立 3D 卡片翻轉效果
- 使用動畫套件中預先建構的精美動畫效果
- 新增對最新版 Android 提供的 *預測返回* 手勢支援

課程內容

在本程式碼研究室中，您將瞭解：

- 如何使用**隱含動畫效果**，以少量程式碼製作美觀的動畫。
- 如何使用**明確動畫效果**，透過預先建構的動畫小工具 (例如 `AnimatedSwitcher` 或 `AnimationController`) 設定自己的效果。
- 如何使用 `AnimationController` 定義自己的小工具，顯示 3D 效果。
- 如何使用 `animations` 套件，以最少的設定顯示精美動畫效果。

軟硬體需求

- Flutter SDK
 - IDE，例如 VSCode 或 Android Studio / IntelliJ
-

2. 設定 Flutter 開發環境

如要完成本實驗室，您需要兩項軟體：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室：

- 連線至電腦並設為開發人員模式的實體 [Android](#) (建議使用，以便在步驟 7 中導入預測返回手勢) 或 [iOS](#) 裝置。
- [iOS 模擬器](#) (需要安裝 Xcode 工具)。
- [Android Emulator](#) (需在 Android Studio 中設定)。
- 瀏覽器 (偵錯時必須使用 Chrome)。
- [Windows](#)、[Linux](#) 或 [macOS](#) 桌上型電腦。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如要瞭解作業系統專屬需求，請參閱 docs.flutter.dev/desktop。

驗證安裝

如要確認 Flutter SDK 設定正確，且您已安裝至少一個上述目標平台，請使用 Flutter Doctor 工具：

```
$ flutter doctor
```

```
Doctor summary (to see all details, run flutter doctor -v):
```

```
[✓] Flutter (Channel stable, 3.24.2, on macOS 14.6.1 23G93 darwin-arm64, locale en)
[✓] Android toolchain - develop for Android devices
[✓] Xcode - develop for iOS and macOS
[✓] Chrome - develop for the web
[✓] Android Studio
[✓] IntelliJ IDEA Ultimate Edition
[✓] VS Code
[✓] Connected device (4 available)
[✓] Network resources
```

```
• No issues found!
```

步驟 3 · 約 5 分鐘

3. 執行範例應用程式

下載範例應用程式

使用 `git` 從 GitHub 的 `flutter/samples` 存放區複製開始應用程式。

```
git clone https://github.com/flutter/codelabs.git
cd codelabs/animations/step_01/
```

或者，您也可以[將原始碼下載為 ZIP 檔案](#)。

執行應用程式

如要執行應用程式，請使用 `flutter run` 指令並指定目標裝置，例如 `android`、`ios` 或 `chrome`。如需支援的平台完整清單，請參閱「[支援的平台](#)」頁面。

```
flutter run -d android
```

注意：如要在步驟 7 中查看「預測返回」，請在 Android 裝置或模擬器上執行這個程式碼研究室。

您也可以使用偏好的 IDE 執行及偵錯應用程式。詳情請參閱官方 Flutter 說明文件。

- [Visual Studio Code](#)
- [Android Studio / IntelliJ](#)

瀏覽程式碼

範例應用程式是多重選擇測驗遊戲，包含兩個畫面，並遵循模型-檢視畫面-檢視模型 (MVVM) 設計模式。`QuestionScreen` (View) 會使用 `QuizViewModel` (View-Model) 類別，向使用者詢問 `QuestionBank` (Model) 類別中的多項選擇題。

- `home_screen.dart` - 顯示含有「New Game」(新遊戲) 按鈕的畫面
- `main.dart` - 設定 `MaterialApp` 使用 Material 3 並顯示主畫面
- `model.dart`：定義整個應用程式使用的核心類別
- `question_screen.dart` - 顯示測驗遊戲的 UI
- `view_model.dart`：儲存測驗遊戲的狀態和邏輯，由 `QuestionScreen` 顯示。



目前應用程式不支援任何動畫效果，但使用者按下「New Game」按鈕時，Flutter 的 `Navigator` 類別會顯示預設的檢視畫面轉換效果。

4. 使用隱含動畫效果

隱含動畫不需要任何特殊設定，因此在許多情況下都是絕佳選擇。在本節中，您將更新 `StatusBar` 小工具，使其顯示動畫計分板。如要尋找常見的隱含動畫效果，請瀏覽 [ImplicitlyAnimatedWidget](#) API 說明文件。



建立未加入動畫效果的賽事比分小工具

建立新檔案 `lib/scoreboard.dart`，並加入以下程式碼：

`lib/scoreboard.dart`

```
import 'package:flutter/material.dart';

class Scoreboard extends StatelessWidget {
  final int score;
  final int totalQuestions;

  const Scoreboard({
    super.key,
    required this.score,
    required this.totalQuestions,
  });

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(8.0),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          for (var i = 0; i < totalQuestions; i++)
            Icon(
              Icons.star,
              size: 50,
              color: score < i + 1
                ? Colors.grey.shade400
                : Colors.yellow.shade700,
            ),
        ],
      ),
    );
  }
}
```


然後在 `StatusBar` 小工具的子項中新增 `Scoreboard` 小工具，取代先前顯示分數和問題總數的 `Text` 小工具。編輯器應會自動在檔案頂端加入必要的 `import "scoreboard.dart"`。

lib/question_screen.dart

```
class StatusBar extends StatelessWidget {
  final QuizViewModel viewModel;

  const StatusBar({required this.viewModel, super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 4,
      child: Padding(
        padding: EdgeInsets.all(8.0),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceAround,
          children: [
            Scoreboard(                                     // NEW
              score: viewModel.score,                       // NEW
              totalQuestions: viewModel.totalQuestions,     // NEW
            ),
          ],
        ),
      ),
    );
  }
}
```

這項小工具會顯示每道問題的星號圖示。答對問題後，系統會立即點亮另一顆星星，不會顯示任何動畫。在下列步驟中，您將以動畫方式呈現分數的大小和顏色，通知使用者分數已變更。

使用隱含動畫效果

建立名為 `AnimatedStar` 的新小工具，使用 `AnimatedScale` 小工具在星星啟動時，將 `scale` 金額從 `0.5` 變更為 `1.0`：

lib/scoreboard.dart

```

import 'package:flutter/material.dart';

class Scoreboard extends StatelessWidget {
  final int score;
  final int totalQuestions;

  const Scoreboard({
    super.key,
    required this.score,
    required this.totalQuestions,
  });

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(8.0),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          for (var i = 0; i < totalQuestions; i++)
            AnimatedStar(isActive: score > i), // Edit this line.
        ],
      ),
    );
  }
}

class AnimatedStar extends StatelessWidget { // Add from here...
  final bool isActive;
  final Duration _duration = const Duration(milliseconds: 1000);
  final Color _deactivatedColor = Colors.grey.shade400;
  final Color _activatedColor = Colors.yellow.shade700;

  AnimatedStar({super.key, required this.isActive});

  @override
  Widget build(BuildContext context) {
    return AnimatedScale(
      scale: isActive ? 1.0 : 0.5,
      duration: _duration,
      child: Icon(
        Icons.star,
        size: 50,
        color: isActive ? _activatedColor : _deactivatedColor,
      ),
    );
  }
} // To here.

```

現在，使用者答對問題時，`AnimatedStar` 小工具會使用隱含動畫更新大小。這裡不會為 `Icon's color` 加上動畫效果，只會為 `scale` 加上動畫，這是由 `AnimatedScale` 小工具完成。



使用 Tween 在兩個值之間進行內插

請注意，`isActive` 欄位變更為 `true` 後，`AnimatedStar` 小工具的颜色會立即變更。

如要實現動畫色彩效果，可以嘗試使用 `AnimatedContainer` 小工具 (這是 `ImplicitlyAnimatedWidget` 的另一個子類別)，因為它可以自動為所有屬性 (包括颜色) 設定動畫效果。很抱歉，小工具必須顯示圖示，而非容器。

您也可以嘗試 `AnimatedIcon`，這個類別會在圖示形狀之間實作轉場效果。不過，`AnimatedIcons` 類別中沒有星號圖示的預設實作方式。

我們將改用 `ImplicitlyAnimatedWidget` 的另一個子類別 `TweenAnimationBuilder`，並將 `Tween` 做為參數。`Tween` 類別會採用兩個值 (`begin` 和 `end`) 並計算中間值，以便動畫顯示這些值。在本例中，我們會使用 `ColorTween`，這符合建構動畫效果所需的 `Tween` 介面。

選取 `Icon` 小工具，並在 IDE 中使用「Wrap with Builder」快速動作，將名稱變更為 `TweenAnimationBuilder`。然後提供時間長度和 `ColorTween`。

lib/scoreboard.dart

```
class AnimatedStar extends StatelessWidget {
  final bool isActive;
  final Duration _duration = const Duration(milliseconds: 1000);
  final Color _deactivatedColor = Colors.grey.shade400;
  final Color _activatedColor = Colors.yellow.shade700;

  AnimatedStar({super.key, required this.isActive});

  @override
  Widget build(BuildContext context) {
    return AnimatedScale(
      scale: isActive ? 1.0 : 0.5,
      duration: _duration,
      child: TweenAnimationBuilder(                                // Add from here...
        duration: _duration,
        tween: ColorTween(
          begin: _deactivatedColor,
          end: isActive ? _activatedColor : _deactivatedColor,
        ),
        builder: (context, value, child) {                        // To here.
          return Icon(Icons.star, size: 50, color: value);      // And modify this line.
        },
      ),
    );
  }
}
```

現在，請熱重載應用程式，查看新的動畫。

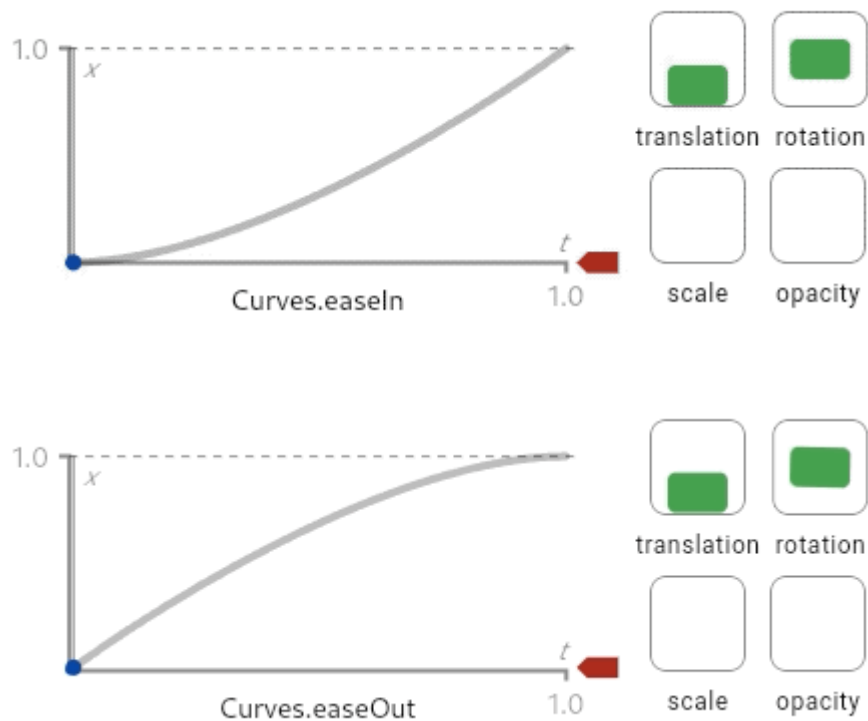


請注意，`ColorTween` 的 `end` 值會根據 `isActive` 參數的值而變更。這是因為每當 `Tween.end` 值有所變更，`TweenAnimationBuilder` 就會重新執行動畫。發生這種情況時，新動畫會從目前的動畫值執行至新的結束值，因此您可以隨時變更顏色 (即使動畫正在執行)，並顯示具有正確中間值的流暢動畫效果。

套用曲線

這兩種動畫效果的速率都保持不變，但動畫加速或減速時，通常會更具視覺吸引力，也更具參考價值。

`Curve` 會套用緩和函式，定義參數隨時間變化的速率。Flutter 隨附 `Curves` 類別中預先建構的緩和曲線集合，例如 `easeIn` 或 `easeOut`。



這些圖表 (位於 `curves` API 說明文件頁面) 說明瞭曲線的運作方式。曲線會將介於 0.0 和 1.0 之間的輸入值 (顯示在 x 軸上) 轉換為介於 0.0 和 1.0 之間的輸出值 (顯示在 y 軸上)。這些圖表也會顯示使用緩和曲線時，各種動畫效果的預覽畫面。

在 `AnimatedStar` 中建立名為 `_curve` 的新欄位，並將其做為參數傳遞至 `AnimatedScale` 和 `TweenAnimationBuilder` 小工具。

[lib/scoreboard.dart](#)

```

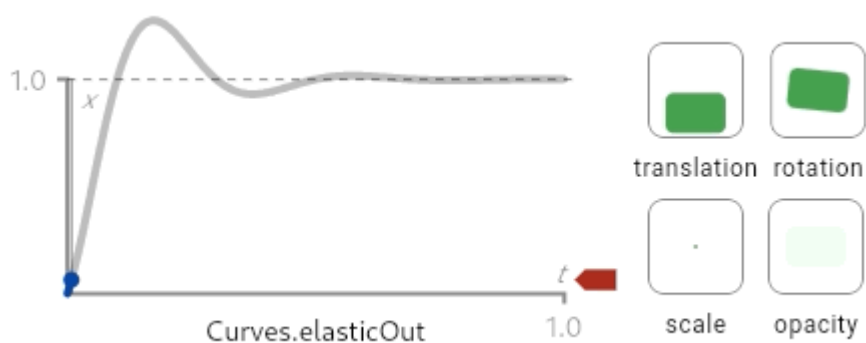
class AnimatedStar extends StatelessWidget {
  final bool isActive;
  final Duration _duration = const Duration(milliseconds: 1000);
  final Color _deactivatedColor = Colors.grey.shade400;
  final Color _activatedColor = Colors.yellow.shade700;
  final Curve _curve = Curves.elasticOut; // NEW

  AnimatedStar({super.key, required this.isActive});

  @override
  Widget build(BuildContext context) {
    return AnimatedScale(
      scale: isActive ? 1.0 : 0.5,
      curve: _curve, // NEW
      duration: _duration,
      child: TweenAnimationBuilder(
        curve: _curve, // NEW
        duration: _duration,
        tween: ColorTween(
          begin: _deactivatedColor,
          end: isActive ? _activatedColor : _deactivatedColor,
        ),
        builder: (context, value, child) {
          return Icon(Icons.star, size: 50, color: value);
        },
      ),
    );
  }
}

```

在本例中，`elasticOut` 曲線會提供誇大的彈簧效果，一開始是彈簧運動，最後會趨於平衡。



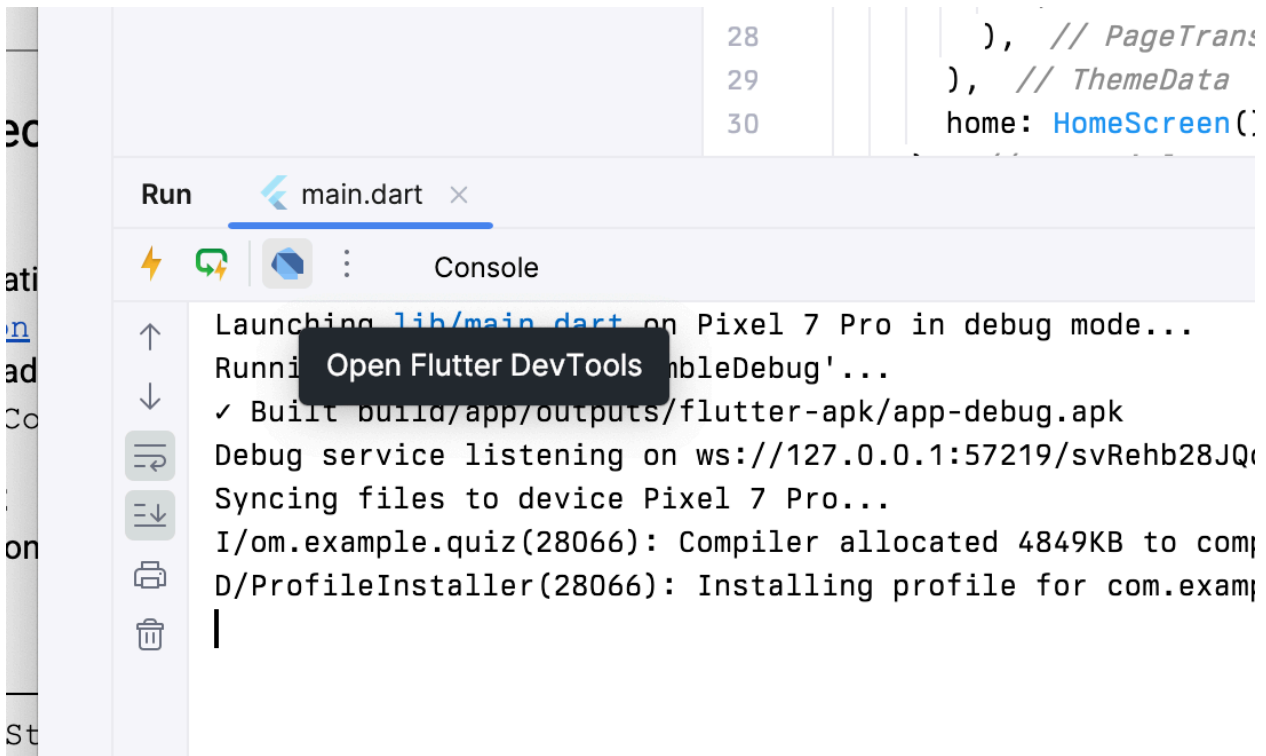
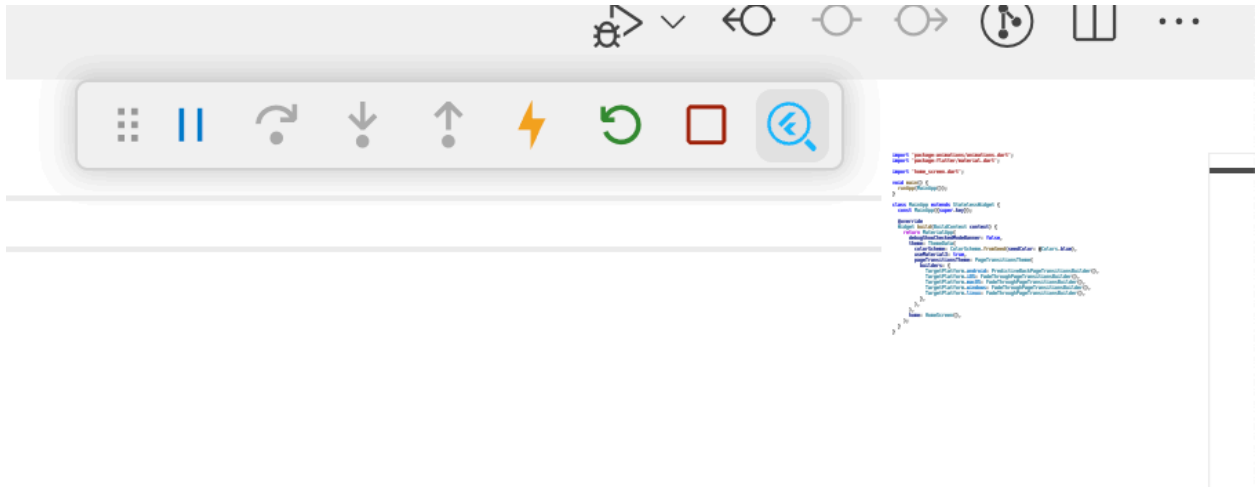
熱重載應用程式，即可看到套用至 `AnimatedSize` 和 `TweenAnimationBuilder` 的曲線。



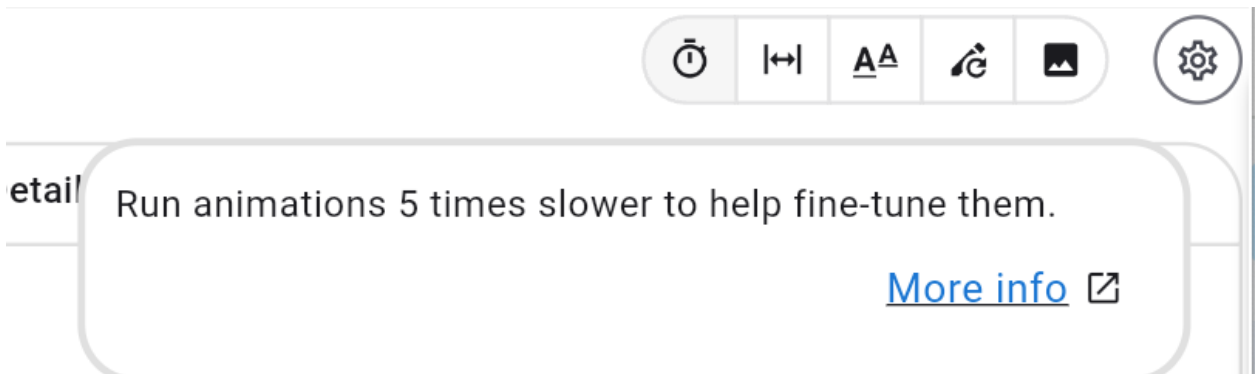
使用開發人員工具啟用慢速動畫

如要偵錯任何動畫效果，Flutter 開發人員工具可放慢應用程式中的所有動畫，方便您更清楚地查看動畫。

如要開啟開發人員工具，請確認應用程式是以偵錯模式執行，然後在 VSCode 的「Debug toolbar」中選取「Widget Inspector」，或在 IntelliJ / Android Studio 的「Debug tool window」中選取「Open Flutter DevTools」按鈕。



開啟小工具檢查器後，按一下工具列中的「Slow animations」按鈕。



5. 使用明確的動畫效果

與隱含動畫類似，明確動畫也是預先建構的動畫效果，但明確動畫會將 `Animation` 物件做為參數，而非採用目標值。舉例來說，如果動畫已由導覽轉場、`AnimatedSwitcher` 或 `AnimationController` 定義，這些函式就非常實用。

使用明確的動畫效果

若要開始使用明確的動畫效果，請使用 包裝 `Card` 小工具。 `Card AnimatedSwitcher`

`lib/question_screen.dart`

```
class QuestionCard extends StatelessWidget {
  final String? question;

  const QuestionCard({required this.question, super.key});

  @override
  Widget build(BuildContext context) {
    return AnimatedSwitcher( // NEW
      duration: const Duration(milliseconds: 300), // NEW
      child: Card(
        key: ValueKey(question),
        elevation: 4,
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Text(
            question ?? '',
            style: Theme.of(context).textTheme.displaySmall,
          ),
        ),
      ), // NEW
    );
  }
}
```

注意：別忘了為 `Card` 小工具提供鍵，這樣 `AnimatedSwitcher` 就能知道何時變更！

`AnimatedSwitcher` 預設會使用淡入淡出效果，但您可以使用 `transitionBuilder` 參數覆寫這項設定。轉場效果建構工具會提供傳遞至 `AnimatedSwitcher` 的子項小工具，以及 `Animation` 物件。這是使用明確動畫的絕佳機會。

在本程式碼研究室中，我們使用的第一個明確動畫是 `SlideTransition`，這個動畫會採用 `Animation<Offset>`，定義傳入和傳出小工具移動時的開始和結束偏移。

`Tween` 有輔助函式 `animate()`，可將任何 `Animation` 轉換為套用 `Tween` 的另一個 `Animation`。也就是說，`Tween` 可用於將 `AnimatedSwitcher` 提供的 `Animation` 轉換為 `Animation`，並提供給 `SlideTransition` 小工具。

`lib/question_screen.dart`


```

class QuestionCard extends StatelessWidget {
  final String? question;

  const QuestionCard({required this.question, super.key});

  @override
  Widget build(BuildContext context) {
    return AnimatedSwitcher(
      transitionBuilder: (child, animation) { // Add from here...
        final curveAnimation = CurveTween(
          curve: Curves.easeInCubic,
        ).animate(animation);
        final offsetAnimation = Tween<Offset>(
          begin: Offset(-0.1, 0.0),
          end: Offset.zero,
        ).animate(curveAnimation);
        return SlideTransition(position: offsetAnimation, child: child);
      }, // To here.
      duration: const Duration(milliseconds: 300),
      child: Card(
        key: ValueKey(question),
        elevation: 4,
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Text(
            question ?? '',
            style: Theme.of(context).textTheme.displaySmall,
          ),
        ),
      ),
    );
  }
}

```

請注意，這會使用 `Tween.animate` 將 `Curve` 套用至 `Animation`，然後將 0.0 到 1.0 範圍的 `Tween` 轉換為 `Tween`，在 x 軸上從 -0.1 轉換為 0.0。

或者，`Animation` 類別有 `drive()` 函式，可接收任何 `Tween` (或 `Animatable`)，並轉換為新的 `Animation`。這樣一來，中間值就能「串連」，讓產生的程式碼更簡潔：

lib/question_screen.dart

```

transitionBuilder: (child, animation) {
  var offsetAnimation = animation
    .drive(CurveTween(curve: Curves.easeInCubic))
    .drive(Tween<Offset>(begin: Offset(-0.1, 0.0), end: Offset.zero));
  return SlideTransition(position: offsetAnimation, child: child);
},

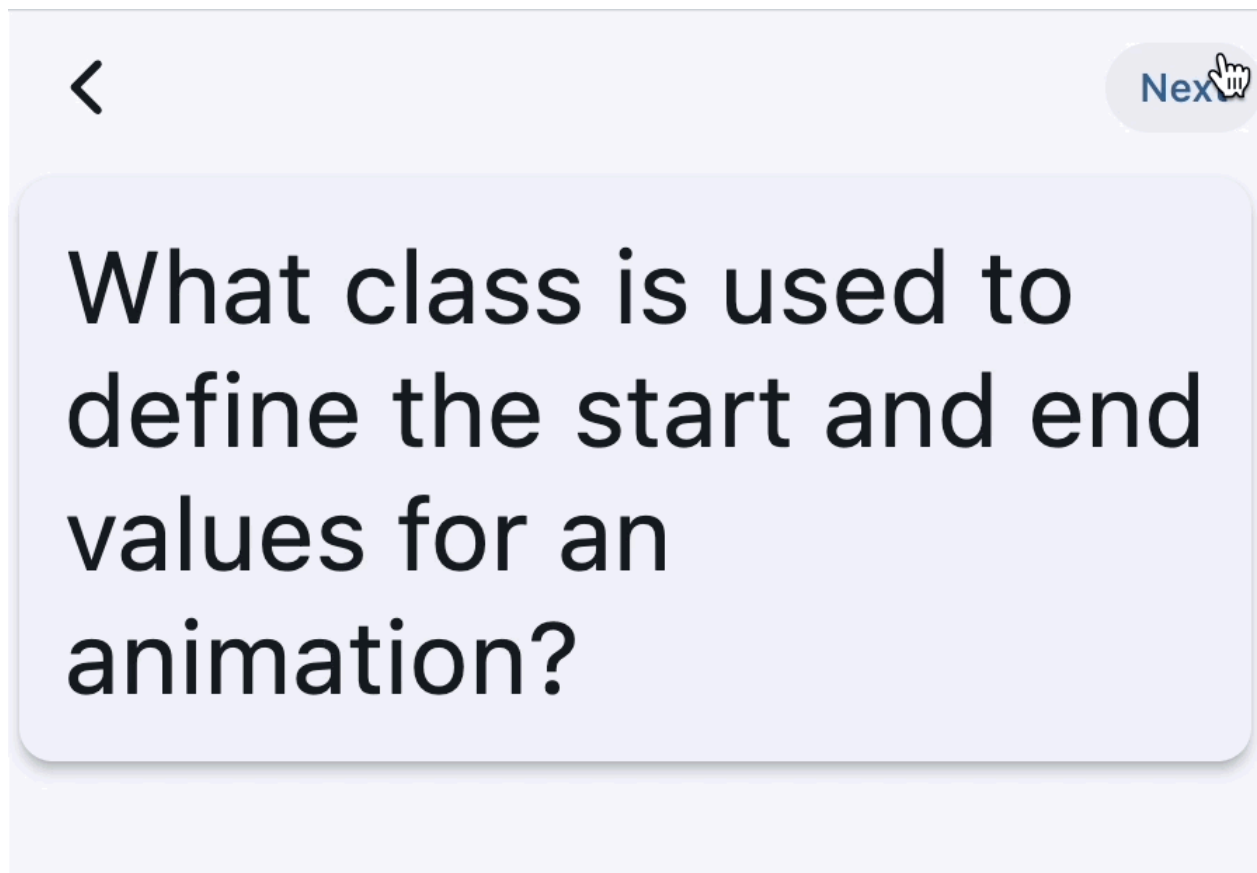
```

使用明確動畫的另一個優點是，這些動畫可以組合在一起。包裝 `SlideTransition` 小工具，新增另一個使用相同曲線動畫的明確動畫 `FadeTransition`。

```
return AnimatedSwitcher(  
  transitionBuilder: (child, animation) {  
    final curveAnimation = CurveTween(  
      curve: Curves.easeInCubic,  
    ).animate(animation);  
    final offsetAnimation = Tween<Offset>(  
      begin: Offset(-0.1, 0.0),  
      end: Offset.zero,  
    ).animate(curveAnimation);  
    final fadeInAnimation = curveAnimation; // NEW  
    return FadeTransition( // NEW  
      opacity: fadeInAnimation, // NEW  
      child: SlideTransition(position: offsetAnimation, child: child), // NEW  
    ); // NEW  
  },  
);
```

自訂 layoutBuilder

你可能會發現「`AnimationSwitcher`」有小問題。`QuestionCard` 切換至新問題時，系統會在動畫執行期間將問題排版在可用空間中央，但動畫停止時，小工具會貼齊畫面頂端。這會導致動畫不穩定，因為問題卡片的最終位置與動畫執行時的位置不符。



如要修正這個問題，`AnimatedSwitcher` 也提供 `layoutBuilder` 參數，可用於定義版面配置。使用這個函式設定版面配置建構工具，將資訊卡對齊畫面頂端：

lib/question_screen.dart

```
@override
Widget build(BuildContext context) {
  return AnimatedSwitcher(
    layoutBuilder: (currentChild, previousChildren) {
      return Stack(
        alignment: Alignment.topCenter,
        children: <Widget>[
          ...previousChildren,
          if (currentChild != null) currentChild,
        ],
      );
    },
  ),
},
```

這段程式碼是 `AnimatedSwitcher` 類別中 `defaultLayoutBuilder` 的修改版本，但使用 `Alignment.topCenter` 而非 `Alignment.center`。

摘要

- 明確動畫是指採用 `Animation` 物件的動畫效果 (與採用目標 `value` 和 `duration` 的 `ImplicitlyAnimatedWidgets` 相對)
 - `Animation` 類別代表正在執行的動畫，但不會定義特定效果。
 - 使用 `Tween().animate` 或 `Animation.drive()` 將 `Tweens` 和 `Curves` (使用 `CurveTween`) 套用至動畫。
 - 使用 `AnimatedSwitcher` 的 `layoutBuilder` 參數調整子項的版面配置方式。
-

6. 控制動畫狀態

到目前為止，所有動畫都是由架構自動執行。隱含動畫會自動執行，而明確動畫效果則需要 `Animation` 才能正常運作。在本節中，您將瞭解如何使用 `AnimationController` 建立自己的 `Animation` 物件，以及如何使用 `TweenSequence` 將 `Tween` 組合在一起。

使用 `AnimationController` 執行動畫

如要使用 `AnimationController` 建立動畫，請按照下列步驟操作：

1. 建立 `StatefulWidget`
2. 在 `State` 類別中使用 `SingleTickerProviderStateMixin` mixin，為 `AnimationController` 提供 `Ticker`
3. 在 `initState` 生命週期方法中初始化 `AnimationController`，並將目前的 `State` 物件提供給 `vsync` (`TickerProvider`) 參數。
4. 請確保每當 `AnimationController` 通知接聽程式時，小工具都會重建，方法是使用 `AnimatedBuilder` 或手動呼叫 `listen()` 和 `setState`。

建立新檔案 `flip_effect.dart`，然後複製並貼上下列程式碼：

[lib/flip_effect.dart](#)

```

import 'dart:math' as math;

import 'package:flutter/widgets.dart';

class CardFlipEffect extends StatefulWidget {
  final Widget child;
  final Duration duration;

  const CardFlipEffect({
    super.key,
    required this.child,
    required this.duration,
  });

  @override
  State<CardFlipEffect> createState() => _CardFlipEffectState();
}

class _CardFlipEffectState extends State<CardFlipEffect>
  with SingleTickerProviderStateMixin {
  late final AnimationController _animationController;
  Widget? _previousChild;

  @override
  void initState() {
    super.initState();

    _animationController = AnimationController(
      vsync: this,
      duration: widget.duration,
    );

    _animationController.addListener(() {
      if (_animationController.value == 1) {
        _animationController.reset();
      }
    });
  }

  @override
  void didUpdateWidget(covariant CardFlipEffect oldWidget) {
    super.didUpdateWidget(oldWidget);

    if (widget.child.key != oldWidget.child.key) {
      _handleChildChanged(widget.child, oldWidget.child);
    }
  }

  void _handleChildChanged(Widget newChild, Widget previousChild) {
    _previousChild = previousChild;
    _animationController.forward();
  }
}

```

```

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: _animationController,
    builder: (context, child) {
      return Transform(
        alignment: Alignment.center,
        transform: Matrix4.identity()
          ..rotateX(_animationController.value * math.pi),
        child: _animationController.isAnimating
          ? _animationController.value < 0.5
            ? _previousChild
            : Transform.flip(flipY: true, child: child)
          : child,
      );
    },
    child: widget.child,
  );
}

```

這個類別會設定 `AnimationController`，並在架構呼叫 `didUpdateWidget` 通知小工具設定已變更，且可能有新的子項小工具時，重新執行動畫。

`AnimatedBuilder` 可確保每當 `AnimationController` 通知事件監聽器時，小工具樹狀結構就會重建，而 `Transform` 小工具則用於套用 3D 旋轉效果，模擬卡片翻轉。

注意：如果是 2D 旋轉，隱含動畫小工具 (`AnimatedRotation`) 或明確版本 (`RotationTransition`) 都可以，但由於沒有預先建構的 `AnimatedTransform` 效果，這個範例會使用 `AnimatedBuilder` 和 `Transform` 小工具。

如要使用這個小工具，請在每個答案資訊卡中加入 `CardFlipEffect` 小工具。請務必為 `Card` 小工具提供 `key`：

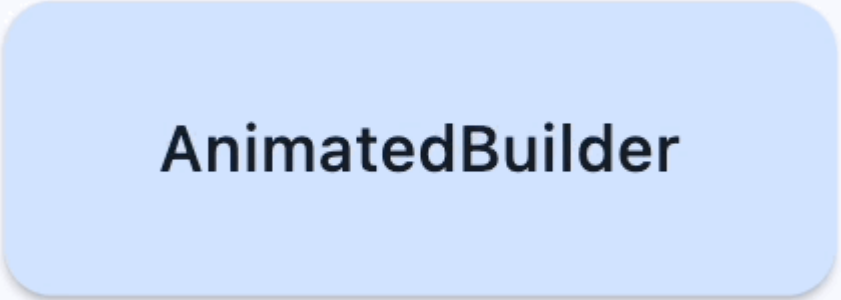
[lib/question_screen.dart](#)

```

@override
Widget build(BuildContext context) {
  return GridView.count(
    shrinkWrap: true,
    crossAxisCount: 2,
    childAspectRatio: 5 / 2,
    children: List.generate(answers.length, (index) {
      var color = Theme.of(context).colorScheme.primaryContainer;
      if (correctAnswer == index) {
        color = Theme.of(context).colorScheme.tertiaryContainer;
      }
      return CardFlipEffect( // NEW
        duration: const Duration(milliseconds: 300), // NEW
        child: Card.filled( // NEW
          key: ValueKey(answers[index]), // NEW
          color: color,
          elevation: 2,
          margin: EdgeInsets.all(8),
          clipBehavior: Clip.hardEdge,
          child: InkWell(
            onTap: () => onTapped(index),
            child: Padding(
              padding: EdgeInsets.all(16.0),
              child: Center(
                child: Text(
                  answers.length > index ? answers[index] : '',
                  style: Theme.of(context).textTheme.titleMedium,
                  overflow: TextOverflow.clip,
                ),
              ),
            ),
          ),
        ),
      ),
    ),
  );
}

```

現在請熱重載應用程式，查看使用 `CardFlipEffect` 小工具翻轉的答案卡。



AnimatedBuilder

您可能會發現這個類別與明確的動畫效果非常相似。事實上，直接擴充 `AnimatedWidget` 類別來實作自己的版本，通常是個好主意。很抱歉，由於這個類別需要在 `State` 中儲存先前的小工具，因此需要使用 `StatefulWidget`。如要進一步瞭解如何建立自己的明確動畫效果，請參閱 [AnimatedWidget](#) 的 API 說明文件。

使用 TweenSequence 新增延遲

在本節中，您將為 `CardFlipEffect` 小工具新增延遲，讓每張資訊卡逐一翻轉。如要開始使用，請新增名為 `delayAmount` 的欄位。

lib/flip_effect.dart

```
class CardFlipEffect extends StatefulWidget {
  final Widget child;
  final Duration duration;
  final double delayAmount; // NEW

  const CardFlipEffect({
    super.key,
    required this.child,
    required this.duration,
    required this.delayAmount, // NEW
  });

  @override
  State<CardFlipEffect> createState() => _CardFlipEffectState();
}
```

然後將 `delayAmount` 新增至 `AnswerCards` 建構方法。

lib/question_screen.dart

```
@override
Widget build(BuildContext context) {
  return GridView.count(
    shrinkWrap: true,
    crossAxisCount: 2,
    childAspectRatio: 5 / 2,
    children: List.generate(answers.length, (index) {
      var color = Theme.of(context).colorScheme.primaryContainer;
      if (correctAnswer == index) {
        color = Theme.of(context).colorScheme.tertiaryContainer;
      }
      return CardFlipEffect(
        delayAmount: index.toDouble() / 2, // NEW
        duration: const Duration(milliseconds: 300),
        child: Card.filled(
          key: ValueKey(answers[index]),

```

然後在 `_CardFlipEffectState` 中，使用 `TweenSequence` 建立新的 `Animation`，套用延遲。請注意，這不會使用 `dart:async` 程式庫中的任何公用程式，例如 `Future.delayed`。這是因為延遲是動畫的一部分，而非小工具使用

`AnimationController` 時明確控制的項目。在開發人員工具中啟用慢速動畫時，由於使用相同的 `TickerProvider`，因此更容易偵錯動畫效果。

如要使用 `TweenSequence`，請建立兩個 `TweenSequenceItem`，其中一個包含 `ConstantTween`，可將動畫維持在 0 的相對時間長度，另一個則包含從 0.0 到 1.0 的一般 `Tween`。

lib/flip_effect.dart

```
class _CardFlipEffectState extends State<CardFlipEffect>
  with SingleTickerProviderStateMixin {
  late final AnimationController _animationController;
  Widget? _previousChild;
  late final Animation<double> _animationWithDelay; // NEW

  @override
  void initState() {
    super.initState();

    _animationController = AnimationController(
      vsync: this,
      duration: widget.duration * (widget.delayAmount + 1),
    );

    _animationController.addListener(() {
      if (_animationController.value == 1) {
        _animationController.reset();
      }
    });

    _animationWithDelay = TweenSequence<double>([ // Add from here...
      if (widget.delayAmount > 0)
        TweenSequenceItem(
          tween: ConstantTween<double>(0.0),
          weight: widget.delayAmount,
        ),
      TweenSequenceItem(tween: Tween(begin: 0.0, end: 1.0), weight: 1.0),
    ]).animate(_animationController); // To here.
  }
```

最後，在 `build` 方法中，將 `AnimationController` 的動畫替換為新的延遲動畫。

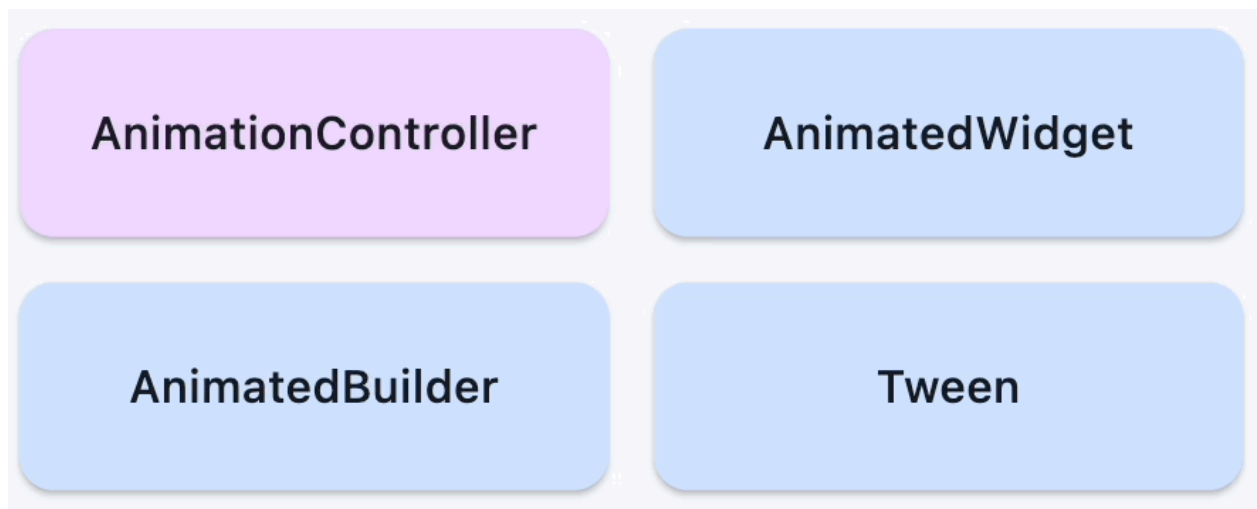
lib/flip_effect.dart

```

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: _animationWithDelay, // Modify this line
    builder: (context, child) {
      return Transform(
        alignment: Alignment.center,
        transform: Matrix4.identity()
          ..rotateX(_animationWithDelay.value * math.pi), // And this line
        child: _animationController.isAnimating
          ? _animationWithDelay.value < 0.5 // And this one.
            ? _previousChild
            : Transform.flip(flipY: true, child: child)
          : child,
      );
    },
    child: widget.child,
  );
}

```

現在請熱重載應用程式，並觀察卡片逐一翻轉。如要挑戰，請試著變更 `Transform` 小工具提供的 3D 效果視角。



7. 使用自訂導覽轉換

到目前為止，我們已瞭解如何自訂單一畫面上的效果，但使用動畫的另一種方式，是利用動畫在畫面之間轉場。在本節中，您將瞭解如何使用內建動畫效果，以及 pub.dev 上官方 [animations](#) 套件提供的精美預先建構動畫效果，為畫面轉換套用動畫效果。

製作導覽轉場動畫

`PageRouteBuilder` 類別是 `Route`，可讓您自訂轉場動畫。您可以覆寫其 `transitionBuilder` 回呼，該回呼會提供兩個 `Animation` 物件，分別代表 `Navigator` 執行的傳入和傳出動畫。

如要自訂轉場動畫，請將 `MaterialPageRoute` 替換為 `PageRouteBuilder`，並在使用者從 `HomeScreen` 瀏覽至 `QuestionScreen` 時自訂轉場動畫。使用 `FadeTransition` (明確的動畫小工具)，讓新畫面在舊畫面上淡入。

`lib/home_screen.dart`

```
ElevatedButton(  
  onPressed: () {  
    // Show the question screen to start the game  
    Navigator.push(  
      context,  
      PageRouteBuilder(  
        // Add from here...  
        pageBuilder: (context, animation, secondaryAnimation) {  
          return const QuestionScreen();  
        },  
        transitionsBuilder:  
          (context, animation, secondaryAnimation, child) {  
            return FadeTransition(  
              opacity: animation,  
              child: child,  
            );  
          },  
      ),  
      // To here.  
    );  
  },  
  child: Text('New Game'),  
),
```

動畫套件提供預先建構的精美動畫效果，例如 `FadeThroughTransition`。匯入動畫套件，並將 `FadeTransition` 替換為 `FadeThroughTransition` 小工具：

`lib/home_screen.dart`

```
import 'package:animations/animations.dart';

ElevatedButton(
  onPressed: () {
    // Show the question screen to start the game
    Navigator.push(
      context,
      PageRouteBuilder(
        pageBuilder: (context, animation, secondaryAnimation) {
          return const QuestionScreen();
        },
        transitionsBuilder:
          (context, animation, secondaryAnimation, child) {
            return FadeThroughTransition( // Add from here...
              animation: animation,
              secondaryAnimation: secondaryAnimation,
              child: child,
            ); // To here.
          },
        ),
    );
  },
  child: Text('New Game'),
),
```

自訂預測返回動畫



預測返回是 Android 的新功能，可讓使用者在導覽前，先查看目前路徑或應用程式後方的內容。當使用者在螢幕上往回拖曳手指時，手指位置會驅動快速瀏覽動畫。

如果 Flutter 的導覽堆疊沒有要彈出的路徑，也就是返回會結束應用程式時，Flutter 會在系統層級啟用這項功能，支援系統預測返回。這項動畫是由系統處理，而非 Flutter 本身。

在 Flutter 應用程式中瀏覽路徑時，Flutter 也支援預測返回手勢。系統會使用名為 `PageTransitionsBuilder` 的特殊 `PredictiveBackPageTransitionsBuilder` 監聽預測返回手勢，並根據手勢進度驅動頁面轉場效果。

預測返回功能僅適用於 Android U 以上版本，但 Flutter 會順利回溯至原始的返回手勢行為和 `ZoomPageTransitionBuilder`。詳情請參閱我們的[網誌文章](#)，包括[設定](#)自有應用程式的相關章節。

在應用程式的 `ThemeData` 設定中，將 `PageTransitionsTheme` 設定為在 Android 上使用 `PredictiveBack`，並在其他平台上使用動畫套件的淡入淡出轉場效果：

lib/main.dart

```
import 'package:animations/animations.dart'; // NEW
import 'package:flutter/material.dart';

import 'home_screen.dart';

void main() {
  runApp(MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        pageTransitionsTheme: PageTransitionsTheme(
          builders: {
            TargetPlatform.android: PredictiveBackPageTransitionsBuilder(), // NEW
            TargetPlatform.iOS: FadeThroughPageTransitionsBuilder(), // NEW
            TargetPlatform.macOS: FadeThroughPageTransitionsBuilder(), // NEW
            TargetPlatform.windows: FadeThroughPageTransitionsBuilder(), // NEW
            TargetPlatform.linux: FadeThroughPageTransitionsBuilder(), // NEW
          },
        ),
      ),
      home: HomeScreen(),
    );
  }
}
```

現在你可以將 `Navigator.push()` 呼叫改回 `MaterialPageRoute`。

lib/home_screen.dart

```
ElevatedButton(  
  onPressed: () {  
    // Show the question screen to start the game  
    Navigator.push(  
      context,  
      MaterialPageRoute(                                // Add from here...  
        builder: (context) {  
          return const QuestionScreen();  
        },  
      ),                                                // To here.  
    );  
  },  
  child: Text('New Game'),  
),
```

使用 FadeThroughTransition 變更目前的問題

`AnimatedSwitcher` 小工具只會在建構工具回呼中提供一個 `Animation`。為解決這個問題，`animations` 套件提供 `PageTransitionSwitcher`。

lib/question_screen.dart

```

class QuestionCard extends StatelessWidget {
  final String? question;

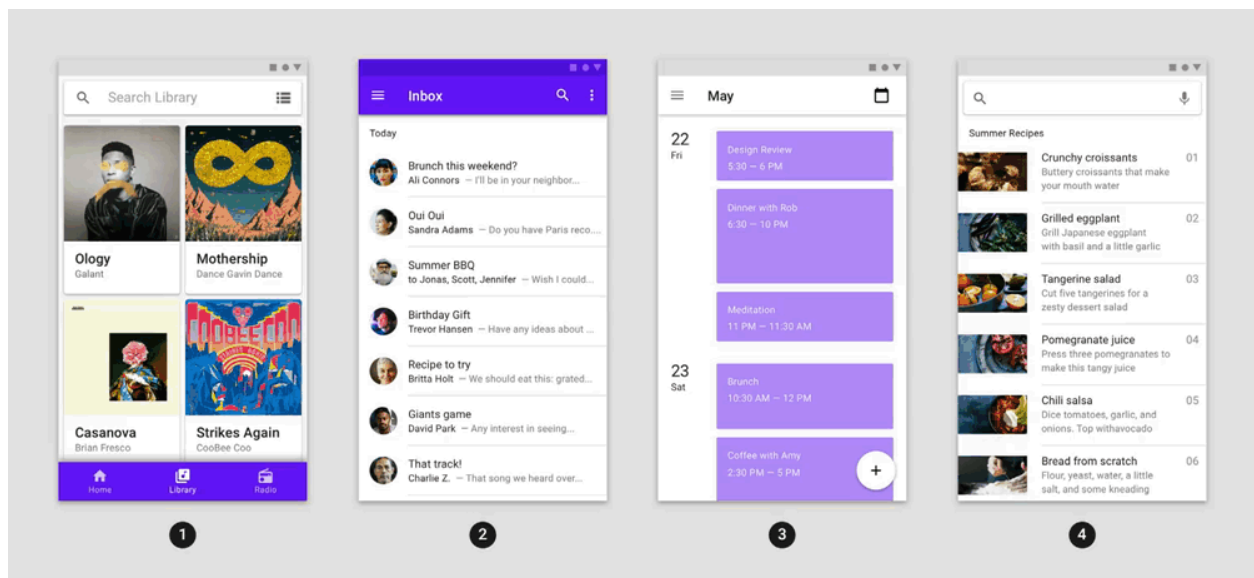
  const QuestionCard({required this.question, super.key});

  @override
  Widget build(BuildContext context) {
    return PageTransitionSwitcher(                                // Add from here...
      layoutBuilder: (entries) {
        return Stack(alignment: Alignment.topCenter, children: entries);
      },
      transitionBuilder: (child, animation, secondaryAnimation) {
        return FadeThroughTransition(
          animation: animation,
          secondaryAnimation: secondaryAnimation,
          child: child,
        );
      },
    ),
    duration: const Duration(milliseconds: 300),
    child: Card(
      key: ValueKey(question),
      elevation: 4,
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(
          question ?? '',
          style: Theme.of(context).textTheme.displaySmall,
        ),
      ),
    ),
  );
}

```

// To here.

使用 OpenContainer



`animations` 套件中的 `OpenContainer` 小工具提供容器轉換動畫效果，可展開並在兩個小工具之間建立視覺連結。

系統一開始會顯示 `closedBuilder` 傳回的小工具，並在輕觸容器或呼叫 `openContainer` 回呼時，展開為 `openBuilder` 傳回的小工具。

如要將 `openContainer` 回呼函式連結至檢視畫面模型，請將 `viewModel` 新增至 `QuestionCard` 小工具，並儲存用於顯示「Game Over」畫面的回呼函式：

[lib/question_screen.dart](#)

```

class QuestionScreen extends StatefulWidget {
  const QuestionScreen({super.key});

  @override
  State<QuestionScreen> createState() => _QuestionScreenState();
}

class _QuestionScreenState extends State<QuestionScreen> {
  late final QuizViewModel viewModel = QuizViewModel(
    onGameOver: _handleGameOver,
  );
  VoidCallback? _showGameOverScreen; // NEW

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: viewModel,
      builder: (context, child) {
        return Scaffold(
          appBar: AppBar(
            actions: [
              TextButton(
                onPressed:
                  viewModel.hasNextQuestion && viewModel.didAnswerQuestion
                    ? () {
                        viewModel.getNextQuestion();
                      }
                    : null,
                child: const Text('Next'),
              ),
            ],
          ),
          body: Center(
            child: Column(
              children: [
                QuestionCard( // NEW
                  onChangeOpenContainer: _handleChangeOpenContainer, // NEW
                  question: viewModel.currentQuestion?.question, // NEW
                  viewModel: viewModel, // NEW
                ), // NEW
                Spacer(),
                AnswerCards(
                  onTap: (index) {
                    viewModel.checkAnswer(index);
                  },
                  answers: viewModel.currentQuestion?.possibleAnswers ?? [],
                  correctAnswer: viewModel.didAnswerQuestion
                    ? viewModel.currentQuestion?.correctAnswer
                    : null,
                ),
                StatusBar(viewModel: viewModel),
              ],
            ),
          ),
        );
      },
    );
  }
}

```

```

        ),
      ),
    );
  },
);
}

void _handleChangeOpenContainer(VoidCallback openContainer) { // NEW
  _showGameOverScreen = openContainer; // NEW
} // NEW

void _handleGameOver() { // NEW
  if (_showGameOverScreen != null) { // NEW
    _showGameOverScreen!(); // NEW
  } // NEW
} // NEW
}

```

新增小工具「GameOverScreen」：

[lib/question_screen.dart](#)

```

class GameOverScreen extends StatelessWidget {
  final QuizViewModel viewModel;
  const GameOverScreen({required this.viewModel, super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(automaticallyImplyLeading: false),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Scoreboard(
              score: viewModel.score,
              totalQuestions: viewModel.totalQuestions,
            ),
            Text('You Win!', style: Theme.of(context).textTheme.displayLarge),
            Text(
              'Score: ${viewModel.score} / ${viewModel.totalQuestions}',
              style: Theme.of(context).textTheme.displaySmall,
            ),
            ElevatedButton(
              child: Text('OK'),
              onPressed: () {
                Navigator.popUntil(context, (route) => route.isFirst);
              },
            ),
          ],
        ),
      ),
    );
  }
}

```

在 `QuestionCard` 小工具中，將 `Card` 替換為 `animations` 套件中的 `OpenContainer` 小工具，並為 `viewModel` 和開啟容器回呼新增兩個欄位：

[lib/question_screen.dart](#)

```

class QuestionCard extends StatelessWidget {
  final String? question;

  const QuestionCard({
    required this.onChangeOpenContainer,
    required this.question,
    required this.viewModel,
    super.key,
  });

  final ValueChanged<VoidCallback> onChangeOpenContainer;
  final QuizViewModel viewModel;

  static const _backgroundColor = Color(0xffff2f3fa);

  @override
  Widget build(BuildContext context) {
    return PageTransitionSwitcher(
      duration: const Duration(milliseconds: 200),
      transitionBuilder: (child, animation, secondaryAnimation) {
        return FadeThroughTransition(
          animation: animation,
          secondaryAnimation: secondaryAnimation,
          child: child,
        );
      },
      child: OpenContainer(
        key: ValueKey(question),
        tappable: false,
        closedColor: _backgroundColor,
        closedShape: const RoundedRectangleBorder(
          borderRadius: BorderRadius.all(Radius.circular(12.0)),
        ),
        closedElevation: 4,
        closedBuilder: (context, openContainer) {
          onChangeOpenContainer(openContainer);
          return ColoredBox(
            color: _backgroundColor,
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Text(
                question ?? '',
                style: Theme.of(context).textTheme.displaySmall,
              ),
            ),
          );
        },
        openBuilder: (context, closeContainer) {
          return GameOverScreen(viewModel: viewModel);
        },
      ),
    );
  }
}

```

```
}  
}
```



Next

What class used to
create custom explicit
animations in Flutter?

AnimationController

AnimatedWidget

AnimatedBuilder

Tween



步驟 8 · 約 0 分鐘

8. 恭喜

恭喜！您已成功在 Flutter 應用程式中新增動畫效果，並瞭解 Flutter 動畫系統的核心元件。具體來說，您學會了：

- 如何使用「`ImplicitlyAnimatedWidget`」
- 如何使用「`ExplicitlyAnimatedWidget`」
- 如何將 `Curves` 和 `Tweens` 套用至動畫
- 如何使用預先建構的轉場效果小工具，例如 `AnimatedSwitcher` 或 `PageRouteBuilder`
- 如何使用 `animations` 套件中預先建構的精美動畫效果，例如 `FadeThroughTransition` 和 `OpenContainer`
- 如何自訂預設轉場動畫，包括在 Android 上新增對預測返回的支援。



後續步驟

請參閱下列程式碼研究室：

- [使用 Material 3 建構動畫式自動調整式應用程式版面配置](#)
- [利用適用於 Flutter 的質感設計動態效果建構精美的轉換效果](#)
- [讓 Flutter 應用程式從無趣變為美觀](#)

您也可以下載[動畫範例應用程式](#)，瞭解各種動畫技巧。

延伸閱讀

如要查看更多動畫資源，請前往 [flutter.dev](#)：

- [動畫簡介](#)
- [動畫教學課程](#) (教學課程)
- [隱含動畫](#) (教學課程)
- [為容器的屬性製作動畫](#) (食譜)
- [淡入和淡出小工具](#) (食譜)
- [主頁橫幅動畫](#)
- [為頁面路徑轉場加上動畫效果](#) (食譜)
- [使用物理模擬製作小工具動畫](#) (食譜)
- [交錯動畫](#)
- [動畫和小工具](#) (小工具目錄)

或參閱 Medium 上的下列文章：

- [動畫深入說明](#)
- [在 Flutter 中自訂隱含動畫](#)
- [使用 Flutter 和 Flux / Redux 管理動畫](#)
- [如何選擇適合自己的 Flutter 動畫小工具？](#)
- [內建明確動畫的定向動畫](#)
- [使用隱含動畫的 Flutter 動畫基礎](#)
- [何時該使用 AnimatedBuilder 或 AnimatedWidget？](#)

參考文件

- [Animations API 總覽](#)
-